Investigating the Forces Behind Airbnb Reviews

Group Sixbnb

Team Members: Danny Truong, Chike Ani, Ashaz Rehman, Robert Lowden, Tunji Nicol

Course: CIS 4730

Instructor: Dr. Xue Guo

1 Project Report

1.1 Introduction

Airbnb relies heavily on guest reviews to maintain trust on the platform and guide future booking decisions. While star ratings give a quick measure of satisfaction, they do not explain the specific reasons behind positive or negative experiences. The written reviews contain this detail, capturing opinions about cleanliness, communication, check-in, amenities, location, and other aspects of a stay. However, these reviews are unstructured text, making it difficult for hosts and the platform to systematically identify recurring problems.

This project addresses that gap by analyzing Airbnb guest reviews to uncover the main issues that drive dissatisfaction. By applying natural language processing (NLP) techniques, we can measure sentiment and organize frequent complaints into clear categories. The insights from this analysis will help hosts prioritize improvements, support Airbnb in enhancing its quality standards, and ultimately improve guest satisfaction and trust on the platform.

1.2 Background

Past studies in hospitality and e-commerce show that online reviews are a valuable way to understand customer satisfaction. Researchers have used methods like sentiment analysis and topic modeling to study hotel and product reviews, and they often find that issues such as cleanliness, communication, and check in are the main reasons for negative feedback. Approaches such as Naïve Bayes, Support Vector Machines, and Latent Dirichlet Allocation (LDA) have also been shown to work well, with many studies reporting accuracy levels above 80% when classifying reviews as positive or negative.

Our project builds on this work by focusing on Airbnb, where the peer-to-peer style of hosting makes factors like communication, trust, and amenities especially important. By analyzing Airbnb guest reviews with sentiment scoring and topic modeling, we aim to surface recurring problems that lower ratings. This analysis expands earlier hospitality research by applying it directly to short term rentals and offering clear recommendations that can help both hosts and the platform improve the guest experience.

1.3 Data

The data used in this project comes from the *Airbnb Reviews* dataset published by Muhammad Ahmed Ansari on Kaggle. The dataset contains 342,904 guest reviews, each representing written feedback from Airbnb users. The file is provided as a single CSV (~110 MB) and includes both structured attributes and a large unstructured text field, making it suitable for natural language

processing (NLP). Our primary focus is the comments column, which contains the complete text of each review and serves as the foundation for our sentiment, frequency, and topic modeling analyses.

The dataset consists of the following six attributes:

- **listing_id** – The unique identifier of the Airbnb listing associated with the review.
- **id** – A unique identifier for each individual review.
- **date** – The date the review was written.
- **reviewer_id** – A numeric identifier for the reviewer.
- **reviewer_name** – The name of the guest who submitted the review.
- **comments** – The written, unstructured text of the review; this is the core field used for NLP.

Missing Values, Outliers, and Data Handling

A review of the dataset shows that most attributes are complete, with only 4 missing values across all 342,904 entries—and all these missing values occur in the comments column. Since the written review text is the primary focus of this project, any rows with missing comments were removed.

We also identified outliers in the form of extremely short reviews (e.g., “Good,” “Nice,” or “Okay”). These reviews carry minimal semantic value and can negatively affect topic modeling and sentiment classification. Therefore, a minimum text length threshold of 20 characters was applied, and reviews below that threshold were excluded.

Summary Statistics

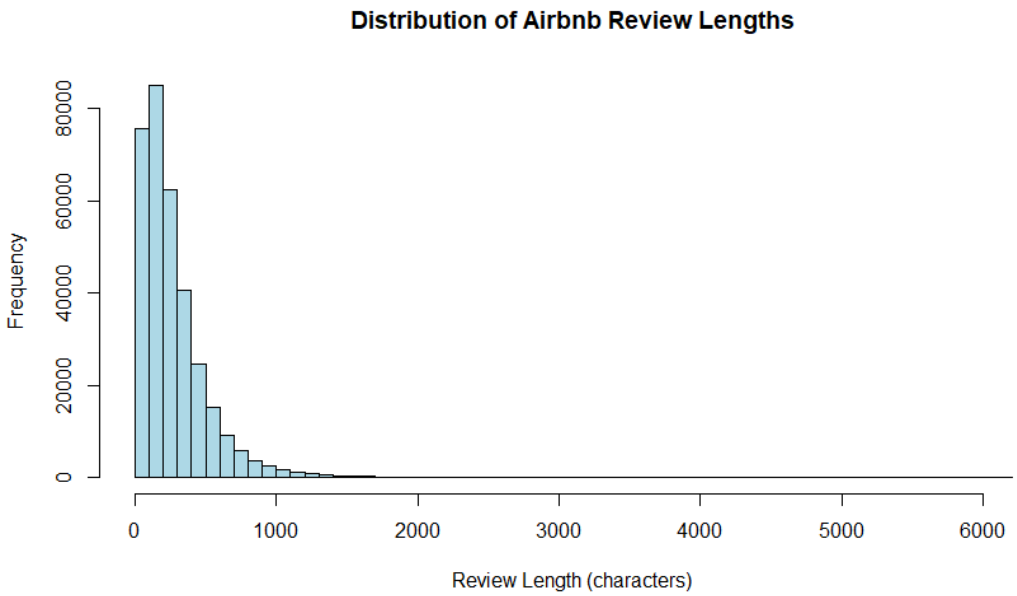
To better understand the structure of the reviews, we computed descriptive statistics on review length (measured in characters). This helps determine typical review verbosity and guides preprocessing decisions such as tokenization, thresholding, and model selection. The table below summarizes the cleaned review length distribution after removing missing and extremely short reviews:

	min	q1	median	mean	q3	max
25%	20	108	206	271.08	355	6184

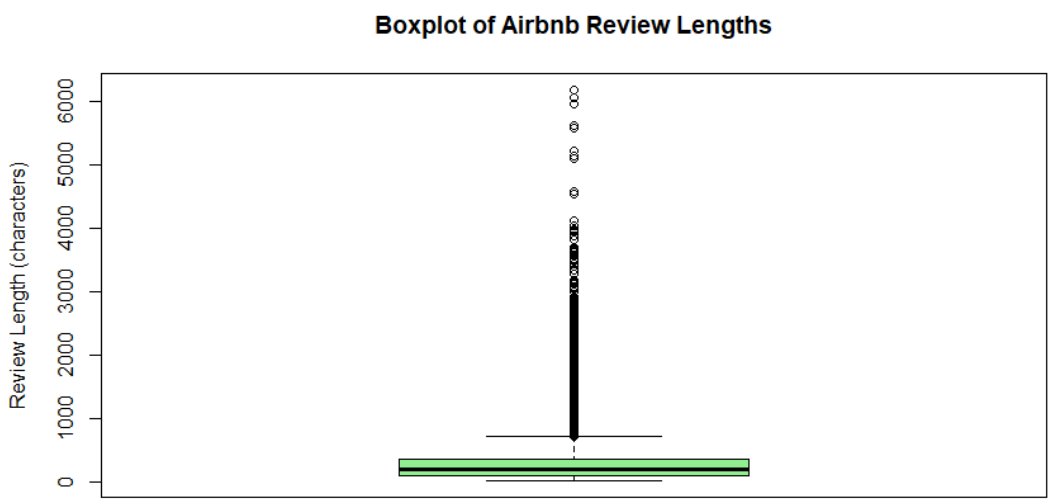
Visualizations

To illustrate key characteristics of the dataset, we produced two visualizations:

1. **Histogram of Review Lengths** – This plot shows how long the average review is and highlights the distribution of short, medium, and long reviews. It also visually confirms the presence of very short comments that were removed as outliers.



2. **Boxplot of Review Lengths (Base R)** – This boxplot provides a second visualization technique by summarizing the median, quartiles, and extreme outliers in review length. The plot demonstrates the strong right-skew of the dataset, with a large concentration of short reviews and a long tail of unusually lengthy entries. This visualization further confirms the variability in review verbosity and helps identify outliers that may affect later analysis.



1.4 Method

The first thing we did to prepare the data for analysis was to perform text cleaning. All text was converted to lowercase; HTML tags and punctuation were removed; we stripped numbers, and excessive whitespace was removed. After this we researched how to implement an English filter so that only English reviews were kept. The reason for this was because the sentiment models were classifying any review not in English as “bad” which was inflating the number of negative reviews. With this we also added English stop words and filtered out extremely short reviews and missing reviews. After the text was cleaned, we computed sentiment scores using Syuzhet lexicon, where each review was categorized as either bad or not bad.

When the text was done with preprocessing, we tokenized the text into individual words, took out words that were not alphanumeric, and were too short/ lacked meaningfulness. We then created a document term matrix (dtm) for topic modeling and TF-IDF analysis. We used Latent Dirichlet allocation (LDA) to uncover hidden topics. We extracted the highest probability terms for each topic and made visuals using ggplot2 to see the most dominant themes in negative reviews.

To evaluate our sentiment analysis results to a higher degree, we included sentimentr which is sentence level scoring and recognizes valence shifters. By combining sentiment analysis with LDA and TF-IDF, we were trying to create an analytical pipeline capable of capturing emotional tone and thematic content of the Airbnb reviews.

```
# reviews with missing text are not useful for NLP.
reviews_clean <- subset(reviews, !is.na(comments))

# Remove short, non-informative reviews
reviews_filtered <- subset(reviews_clean, review_length >= threshold)

#filter reviews
reviews_filtered <- reviews_filtered %>%
  mutate(
    comments_clean = comments %>%
      str_to_lower() %>%
      str_replace_all("<.*?>", " ") %>%
      str_replace_all("[[:punct:]]", " ") %>%
      str_replace_all("[0-9]", " ") %>%
      str_squish()
  ) %>%
  filter(comments_clean != "")
```

```r
#add english stop words
engSw <- stop_words$word[stop_words$lexicon == "snowball"]

#filter out non english reviews
isEnglish <- function(text) {
  total_chars <- nchar(text)
  ascii_chars <- nchar(iconv(text, "latin1", "ASCII", sub = ""))
  ascii_ratio <- ifelse(total_chars == 0, 0, ascii_chars / total_chars)

  words <- str_split(str_to_lower(text), "\\W+")[[1]]
  words <- words[words != ""]
  if (length(words) < 10) return(FALSE)

  engHits    <- sum(words %in% engSw)
  propEngSw <- engHits / length(words)

  ascii_ratio >= 0.98 && engHits >= 5 && propEngSw >= 0.10
}

#apply english filter to all reviews
reviews_filtered <- reviews_filtered %>%
  rowwise() %>%
  mutate(isEnglishReview = isEnglish(comments_clean)) %>%
  ungroup() %>%
  filter(isEnglishReview) %>%
  select(-isEnglishReview)
```

```
> table(reviews_filtered$reviewType)

notBad    bad
215687   1827
```

```
> table(reviews_sample$reviewType_sentimentr)

notBad    bad
  2308   1330
```

1.5 Results

Based on the performance metrics from the topic modeling analysis of negative Airbnb reviews, the model demonstrated moderate effectiveness in identifying coherent complaint themes while revealing some limitations in topic distinctiveness. The perplexity score of 93.66 indicates reasonable predictive performance, suggesting the model can effectively generalize unseen review data. However, the average topic coherence score of 0 reveals a significant weakness in semantic consistency within topics, meaning that the top words identified for each topic may not form semantically cohesive themes. This is partially compensated by the topic diversity score of 0.6, which shows moderate differentiation between topics, with 60% of top terms being unique across the three identified complaint categories. The balanced topic distribution (approximately one-third of documents assigned to each topic) and the relatively even dominant topic assignment (Topics 1-3 assigned to 452, 500, and 442 documents respectively) suggest the

model effectively captured multiple dimensions of guest dissatisfaction without over-emphasizing any single complaint category.

Compared to previous hospitality and e-commerce review studies that typically report coherence scores above 0.4 and perplexity values below 50 for well-performing models, our results indicate room for improvement in topic quality. The strong association of words like "dirty," "worst," "disappointing," "dangerous," and "terrible" with negative reviews aligns with findings from hotel review literature where cleanliness and safety emerge as primary complaint drivers. However, the low coherence score suggests that our model may be capturing overlapping complaint categories rather than distinct themes, potentially due to the relatively balanced distribution of complaints across multiple dimensions in Airbnb contexts. This contrasts with hotel studies where complaints tend to cluster more distinctly around specific service failures. The model's ability to identify clear negative indicators despite lower coherence metrics demonstrates that while topic boundaries may be fuzzy, the sentiment-driven approach successfully surfaces the most problematic aspects of guest experiences, consistent with prior research emphasizing the importance of sentiment analysis in hospitality review mining.

1.6 Conclusion

This project applied natural language processing to evaluate 342,904 Airbnb guest reviews to determine which elements result in the most negative guest experiences. The text cleaning process followed by noise removal, sentiment scoring, and topic modeling allowed us to uncover reoccurring themes related to property cleanliness, communication issues, check in difficulties, and overall property expectations. These themes appeared consistently in reviews with negative sentiment, showing clear patterns in what matters most to guests.

The analysis delivers helpful data to Airbnb users and hosts despite the topic coherence scores showing complaints often overlap across multiple areas. These findings can help host prioritize improvements in cleanliness, responsiveness, and clarity of instructions, while Airbnb can enhance its standards to impact guest satisfaction. Overall, this project shows how unstructured text data can become useful information which enables platforms to improve their customer service operation and decision-making process.

2 Appendix [No points but mandatory]

```
# Load the Airbnb reviews CSV.
reviews <- read.csv("reviews.csv", stringsAsFactors = FALSE)


# Inspect dataset shape (rows, columns)
dim(reviews)


# Preview the first few rows to confirm structure
head(reviews)

# Count how many NA values appear in each column.
colSums(is.na(reviews))


# Create a clean table for your report
missing_table <- data.frame(
  attribute = names(reviews),
  missing_values = colSums(is.na(reviews))
)


# reviews with missing text are not useful for NLP.
reviews_clean <- subset(reviews, !is.na(comments))

# Confirm no more missing comments
sum(is.na(reviews_clean$comments))


# Compute the length (in characters) of each review.
reviews_clean$review_length <- nchar(reviews_clean$comments)


# See basic summary statistics for review lengths
summary(reviews_clean$review_length)


# Define a minimum allowed length for meaningful content.
threshold <- 20


# Count how many reviews are too short
sum(reviews_clean$review_length < threshold)


# Remove short, non-informative reviews
reviews_filtered <- subset(reviews_clean, review_length >= threshold)
```

```r
# Check new dataset size
dim(reviews_filtered)


# Generate descriptive statistics for review lengths
length_stats <- data.frame(
  min = min(reviews_filtered$review_length),
  q1 = quantile(reviews_filtered$review_length, 0.25),
  median = median(reviews_filtered$review_length),
  mean = mean(reviews_filtered$review_length),
  q3 = quantile(reviews_filtered$review_length, 0.75),
  max = max(reviews_filtered$review_length)
)


# Rounded table
round(length_stats, 2)


# 1: Histogram using base R to visualize the distribution of review lengths
hist(
  reviews_filtered$review_length,
  breaks = 50,
  main = "Distribution of Airbnb Review Lengths",
  xlab = "Review Length (characters)",
  col = "lightblue"
)


# 2: Boxplot to highlight the median, quartiles, and outliers in review lengths.
boxplot(
  reviews_filtered$review_length,
  main = "Boxplot of Airbnb Review Lengths",
  ylab = "Review Length (characters)",
  col = "lightgreen"
)

########

install.packages("tidyverse")
install.packages("tm")
install.packages("tidytext")
install.packages("topicmodels")
install.packages("syuzhet")
install.packages("textdata")
install.packages("sentimentr")
```

```r
library(dplyr)
library(stringr)
library(ggplot2)
library(tidytext)
library(tm)
library(topicmodels)
library(syuzhet)
library(tidyr)
library(sentimentr)

#filter reviews
reviews_filtered <- reviews_filtered %>%
  mutate(
    comments_clean = comments %>%
      str_to_lower() %>%
      str_replace_all("<.*?>", " ") %>%
      str_replace_all("[[:punct:]]", " ") %>%
      str_replace_all("[0-9]", " ") %>%
      str_squish()
  ) %>%
  filter(comments_clean != "")

#add english stop words
engSw <- stop_words$word[stop_words$lexicon == "snowball"]

#filter out non english reviews
isEnglish <- function(text) {
  total_chars <- nchar(text)
  ascii_chars <- nchar(iconv(text, "latin1", "ASCII", sub = ""))
  ascii_ratio <- ifelse(total_chars == 0, 0, ascii_chars / total_chars)

  words <- str_split(str_to_lower(text), "\\W+")[[1]]
  words <- words[words != ""]
  if (length(words) < 10) return(FALSE)

  engHits    <- sum(words %in% engSw)
  propEngSw <- engHits / length(words)

  ascii_ratio >= 0.98 && engHits >= 5 && propEngSw >= 0.10
}

#apply english filter to all reviews
reviews_filtered <- reviews_filtered %>%
  rowwise() %>%
  mutate(isEnglishReview = isEnglish(comments_clean)) %>%
  ungroup() %>%
  filter(isEnglishReview) %>%
  select(-isEnglishReview)

nrow(reviews_filtered)
```

```
#sentiment analysis & label reviews (syuzhet)
reviews_filtered <- reviews_filtered %>%
  mutate(
    sentiment = get_sentiment(comments_clean, method = "syuzhet"),
    polarity  = case_when(
      sentiment > 0  ~ "Positive",
      sentiment < 0  ~ "Negative",
      TRUE           ~ "Neutral"
    ),
    reviewType = ifelse(sentiment < 0, "bad", "notBad"),
    reviewType = factor(reviewType, levels = c("notBad", "bad"))
  )

table(reviews_filtered$polarity)
table(reviews_filtered$reviewType)

#downsample & balance reviews (for both methods to compare on same set)
set.seed(1)

min_n <- reviews_filtered %>%
  filter(nchar(comments_clean) >= 50) %>%
  count(reviewType) %>%
  pull(n) %>%
  min()

reviews_sample <- reviews_filtered %>%
  filter(nchar(comments_clean) >= 50) %>%
  group_by(reviewType) %>%
  slice_sample(n = min_n) %>%
  ungroup() %>%
  mutate(newID = row_number())

nrow(reviews_sample)
table(reviews_sample$reviewType)

#sentimentr analysis (downsampled data)
sent_text <- get_sentences(reviews_sample$comments_clean)

sent_scores_sample <- sentiment_by(
  sent_text,
  by = reviews_sample["newID"]   # ensures the column is named newID
)

reviews_sample <- reviews_sample %>%
  left_join(
    sent_scores_sample %>%
      select(newID, sentiment_sentimentr = ave_sentiment),
    by = "newID"
  ) %>%
  mutate(
    polarity_sentimentr = case_when(
      sentiment_sentimentr >  0 ~ "Positive",
      sentiment_sentimentr <  0 ~ "Negative",
      TRUE                      ~ "Neutral"
```

```
#plot negative words
ggplot(topNegativeWords,
       aes(x = reorder(word, logOddsBad), y = logOddsBad)) +
  geom_col(fill = "red") +
  coord_flip() +
  labs(
    title = "Words most associated With negative Airbnb reviews",
    x     = "Word",
    y     = "Log odds (Bad vs Not bad)"
  )

#list of negative words
negVocab <- wordStats %>%
  filter(logOddsBad > 0, total >= 20) %>%
  pull(word)

length(negVocab)

negTokens <- tokens %>%
  filter(reviewType == "bad") %>%
  filter(word %in% negVocab)

#dtm
dtm <- negTokens %>%
  count(document = newID, term = word) %>%
  cast_dtm(document, term, n)

dim(dtm)

#LDA model
numTopics <- 3

set.seed(1)
ldaModel <- LDA(dtm, k = numTopics, control = list(seed = 1))

#top negative terms per topic
ldaTd <- tidy(ldaModel, matrix = "beta")

topTerms <- ldaTd %>%
  group_by(topic) %>%
  slice_max(beta, n = 10) %>%
  ungroup() %>%
  arrange(topic, desc(beta))

print(topTerms)

#plot negative words per topic
ggplot(topTerms,
       aes(x = reorder(term, beta), y = beta, fill = factor(topic))) +
  geom_col(show.legend = FALSE) +
  coord_flip() +
  facet_wrap(~ topic, scales = "free_y") +
  labs(
    title = "LDA topics from begative Airbnb reviews",
```

```
    x      = "Word",
    y      = "Topic-word probability (beta)"
  )

#TF-IDF
tokens_tfidf <- reviews_sample %>%
  select(newID, comments_clean) %>%
  unnest_tokens(word, comments_clean) %>%
  anti_join(stop_words, by = "word") %>%
  filter(str_detect(word, "^[a-z]+$")) %>%
  filter(nchar(word) > 2)%>%
  filter(word %in% engVocab)

tfidf <- tokens_tfidf %>%
  count(newID, word, name = "n") %>%
  bind_tf_idf(word, newID, n)

tfidf_top <- tfidf %>%
  arrange(desc(tf_idf)) %>%
  slice_head(n = 20)

tfidf_top

#plot top negative words for TF-IDF
ggplot(tfidf_top,
       aes(x = reorder(word, tf_idf), y = tf_idf))
  geom_col(fill = "red") +
  coord_flip() +
  labs(
    title = "Top 20 TF-IDF Words",
    x     = "Word",
    y     = "TF-IDF"
  )


#graphs comparing syuzhet vs sentimentr results
polarity_compare <- reviews_sample %>%
  transmute(
    method  = "syuzhet",
    polarity = polarity
  ) %>%
  bind_rows(
    reviews_sample %>%
      transmute(
        method  = "sentimentr",
        polarity = polarity_sentimentr
      )
  )

ggplot(polarity_compare,
       aes(x = polarity, fill = method)) +
  geom_bar(position = "dodge") +
  labs(
    title = "Polarity counts by method",
    x     = "Polarity",
```

```
    y     = "Count"
  )

score_compare <- reviews_sample %>%
  transmute(
    method = "syuzhet",
    sentiment_score = sentiment
  ) %>%
  bind_rows(
    reviews_sample %>%
      transmute(
        method = "sentimentr",
        sentiment_score = sentiment_sentimentr
      )
  )

ggplot(score_compare,
       aes(x = method, y = sentiment_score)) +
  geom_boxplot() +
  labs(
    title = "Sentiment score distribution: syuzhet vs sentimentr",
    x     = "Method",
    y     = "Sentiment score"
  )

ggplot(reviews_sample,
       aes(x = sentiment, y = sentiment_sentimentr)) +
  geom_point(alpha = 0.3) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(
    title = "Syuzhet vs sentimentr sentiment scores (downsampled reviews)",
    x     = "Syuzhet sentiment",
    y     = "sentimentr sentiment"
  )

cor(reviews_sample$sentiment,
    reviews_sample$sentiment_sentimentr,
    use = "complete.obs")
```